

Genetic Algorithms: String Evolution



-Toptal.com

Course Name: OODP

Name: Sparsh Kumar – RA2511051010013

Class/Section: AN-2

Faculty Name: Dr. S Sindhu

Institution Name: SRMIST KTR

Academic Year: 1st

Abstract

- The Project is based on genetic algorithms, how they use bio mimicry practices in the context of computer science and engineering.
- The process is repeated until either satisfactory fitness value is achieved or we run out of gene pool iterations.
- Problem Statement: Reaching a target string from random gibberish starting point string (of same number of characters).
- The user enters the specific string to evolve the random one into → The algorithm takes a random generated string of same length and selects the one with best fitness value calculated by matching each letter to target if its correct high fitness value ones are selected to be in next generation (elitism) and for cross-segmentation and mutation.
- Technologies Used : Pure C++ and standard Libraries.

Introduction

- **Background:** Genetic algorithms are specifically useful to achieve optimal solutions to problems with multiple variables such as the knapsack problem or optimizing the best route etc.
- In the problem we solved: its getting to a target string from random letters via natural selection simulation
- The Need: Traditional Permutation and Combination solutions would scale exponentially to solve if the number of letters or the number of items to optimize were to increase therefore genetic algorithms reduce the time taken to optimize greatly by selecting the best fitting solution instead of permuting and checking each possible one. And they are highly accurate
- **Objective:** Demonstrate usage of genetic algorithms as a viable optimization solution and the beauty of its biomimicry workings.

- **Scope:** Working with strings and manipulation of them. Simulating the, "survival of the fittest" concept to evolve a random string of characters into a target string through genetic algorithms.

OOP Concepts Used

- using `std::string` and `std::vector`. In C++, these are Classes, and when we create target or population, we are instantiating Objects. We are calling their methods (like `.size()`, `.push_back()`, `.substr()`).

```
for (string child : children) {  
    string mutated_child = mutate(child, mutation_rate);  
    new_population.push_back(mutated_child);  
}
```

- By calling `fitness_function(genome)`, the rest of the code doesn't need to know *how* the score is calculated, just that it gets a number back. This is a very fundamental form of abstraction

```
for (const string& genome : population) {  
    int genome_fitness = fitness_function(genome);  
    total_fitness += genome_fitness;  
    fitness_values.push_back(genome_fitness);  
}
```

Genome(string s) — The Constructor

This is the function that runs automatically the very moment an object is created (e.g., `Genome myGenome("Grah");`). It tells the computer: "To make a Genome, you must give me a string."

: word(s) — The Initializer List

- The colon starts the initializer list. This is the most efficient way to assign values to class variables in C++.
- It takes the input `s` and directly copies it into the class's private member variable `word`

```
class Genome {
private:
    string word;
    int fitness;

public:
    Genome() {
        word = "";
        fitness = 0;
    }

    Genome(string s) : word(s), fitness(0) {
        update_fitness();
    }

    void update_fitness() {
        fitness = 0;
        for (int i = 0; i < word.size(); i++) {
            if (word[i] == target[i]) {
                fitness++;
            }
        }
    }
}
```

{ update_fitness(); } — The Body

Once the variable `word` is set, the code inside the curly braces runs.

- In a Genetic Algorithm, you never want a Genome to exist without a fitness score.
- By calling `update_fitness()` here, we ensure that the second the object is born, it calculates its own "strength" automatically.

Getters and setters are also used in class `Genome`

```
// setter
void set_word(const string& new_word) {
    word = new_word;
    update_fitness();
}

// getter
int get_fitness() const { return fitness; }
string get_word() const { return word; }
```

Encapsulation: The reason we use `private` is to ensure the internal state (like `fitness`) can't be changed without going through a function (like `set_word`) that keeps everything consistent.

- marked `word` and `fitness` as **private**. This is the "gatekeeper" effect.
- **Using Public Methods** (`get_fitness`, `set_word`) to interact with that data.
- **Crucial Logic:** When you call `set_word`, it automatically triggers `update_fitness`. This ensures the internal state of the object is always valid and consistent. The user of the class doesn't have to remember to update the fitness manually; the object "knows" how to take care of itself.

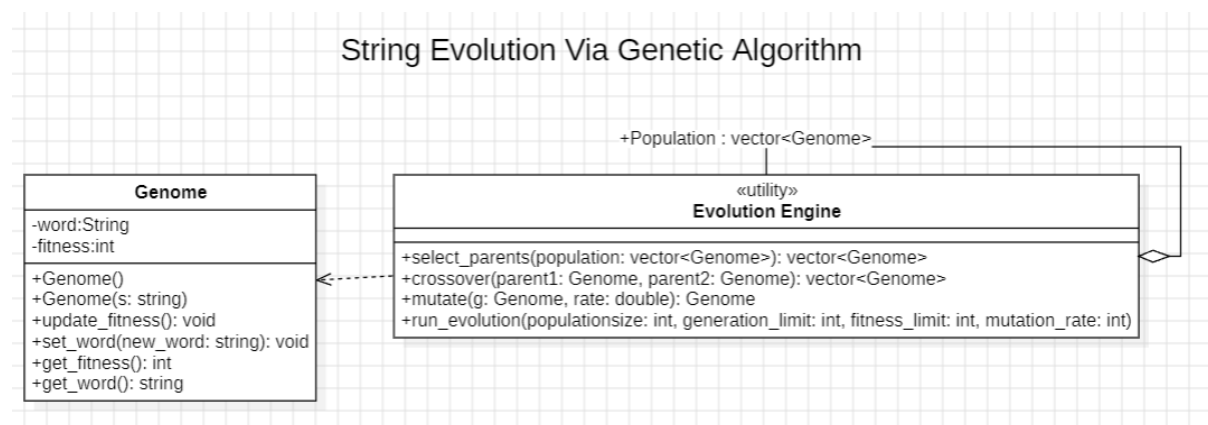
Abstraction

- In `run_evolution()`, we simply call `genome.get_fitness()`. And we don't see the for loop comparing characters or the logic involving the target string. We've **abstracted** the concept of "How good is this string?" into a single, simple function call.

System Analysis

- In the current state, the system only evolves letters (uppercase and lowercase and numbers)
- Limitations: It only evolves good up until 12-15 characters after which it stagnates at a fitting value
- Proposal: Make mutation value increase with fitting numbers
- Advantage of current system: Far better and faster than just raw permutation and combination

System Design



1. Design Philosophy

The system is built using an **Object-Oriented Programming (OOP)** approach to ensure modularity and data integrity. By separating the genetic data from the evolutionary logic, the system is easily extensible—allowing for future changes to the fitness criteria or mutation logic without restructuring the entire codebase.

2. Core Components

- **The Genome Class (Data Model):** Utilizes **Encapsulation** to bundle a string (word) with its calculated fitness score. By making these attributes private, the class ensures that the fitness value is only updated through the `set_word` method, maintaining internal consistency.
- **The Evolution Engine (Utility):** A set of specialized functions that manage the population lifecycle. This includes **Roulette Wheel Selection** for selection pressure,

Single-Point Crossover for genetic recombination, and a probabilistic **Mutation** function to maintain genetic diversity.

3. Evolutionary Lifecycle

The system follows a cyclic process to converge on the target string:

1. **Initialization:** Generates a diverse population of Genome objects with randomized strings.
2. **Selection:** Uses a fitness-proportionate selection (Roulette Wheel) where fitter individuals have a higher statistical probability of becoming parents.
3. **Reproduction:** New offspring are created via crossover, followed by character-wise mutation based on a configurable `mutation_rate`.
4. **Replacement:** The system employs a generational replacement strategy, swapping the old population for the new, fitter generation until the target string is matched.

4. Technical Optimizations

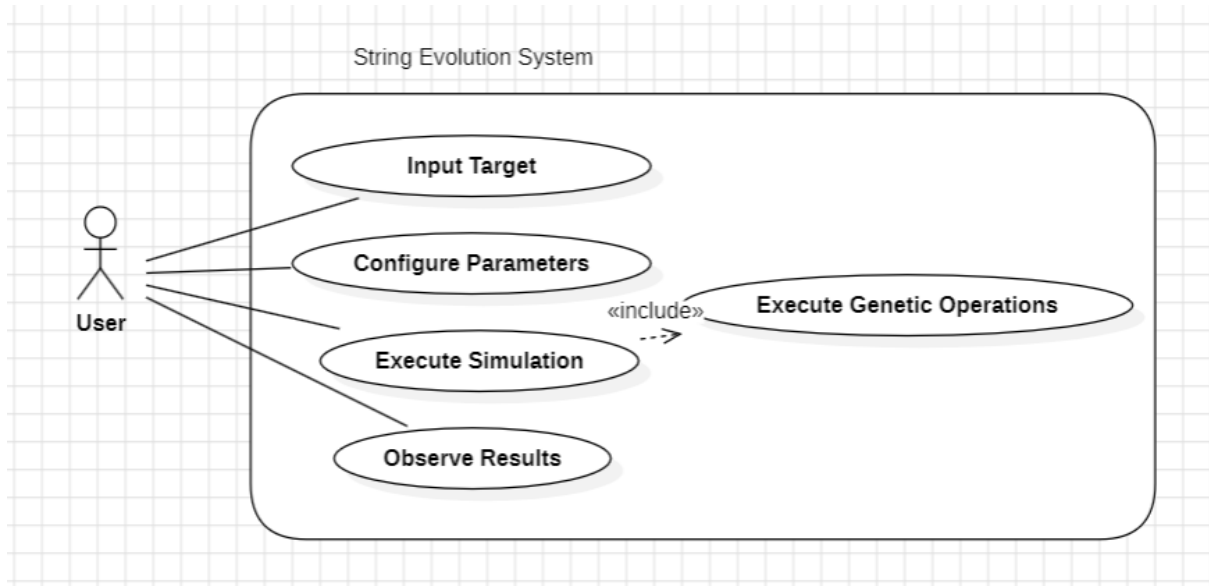
- **Memory Efficiency:** Population is stored in `std::vector` to take advantage of contiguous memory allocation, improving iteration speed.
- **Performance:** Parameters like `const Genome&` are used in function signatures to prevent unnecessary object copying, reducing overhead during large-scale simulations.
- **Randomization:** Implements the `<random>` library's `mt19937` (Mersenne Twister) for high-quality entropy, ensuring the selection and mutation processes remain truly stochastic.

Summary of OOP Concepts Applied

- **Encapsulation:** Hidden state variables (`word`, `fitness`) accessible only via public getters/setters.
- **Abstraction:** The `run_evolution` loop interacts with high-level Genome methods without needing to know the underlying character-comparison logic.

- **Modularity:** The fitness function is localized to the class, making the "goal" of the evolution easy to swap.

Use Case Diagram:



Use Case Description

- Actor: The User (Researcher/Developer).
- System Boundary: The String Evolution Program.
- Use Cases:
 1. Input Target String: User defines the goal.
 2. Configure Parameters: User sets population size and mutation rate.
 3. Monitor Progress: User views real-time generation updates.
 4. Execute Evolution: The core system process (includes Selection, Crossover, and Mutation).

Implementation Details:

- C++ was used to make the whole console application

- Visual Studio 2022 IDE was used
- Standard library modules such as
 - `<iostream>` for input output
 - `<string>` for string functionality
 - `<random>` for randomness
 - `<time.h>` to clock the time taken to execute functions
 - `<list>` for list functionality
 - `<algorithm>` for writing algorithmic code
 - `<vector>` to create vectors for modular sizeable object collections
- Main Modules which work under the hood:
 - Fitness and Selection function
 - Single point Cross over function
 - Mutation function

Output:

```
Welcome to the String Evolution Genetic Algorithm!  
Give me a string to evolve: |
```

```
Welcome to the String Evolution Genetic Algorithm!  
Give me a string to evolve: SRMKTR  
Target String: SRMKTR  
Gen 0 Best: uopw5R (1)  
Gen 1 Best: xR3w5R (2)  
Gen 2 Best: SI9KrR (3)  
Gen 3 Best: SR3wTR (4)  
Gen 4 Best: SRM1TR (5)  
Gen 5 Best: SaMKTL (4)  
Gen 6 Best: SR3KTR (5)  
Gen 7 Best: 2RMKTR (5)  
Gen 8 Best: xRMKTR (5)  
Gen 9 Best: SRMzTR (5)  
Gen 10 Best: SRMKTR (6)  
Genome string evolved: SRMKTR  
Total Evolution Time: 0.007 seconds.
```

```
Welcome to the String Evolution Genetic Algorithm!  
Give me a string to evolve: oodptest  
Target String: oodptest  
Gen 0 Best: odF9eVB (2)  
Gen 1 Best: nohy0eLY (2)  
Gen 2 Best: osdjWTsb (3)  
Gen 3 Best: o3dp9dsn (4)  
Gen 4 Best: oohyesZ (4)  
Gen 5 Best: oodpsgrt (5)  
Gen 6 Best: oodpsgsg (5)  
Gen 7 Best: ooRptnst (6)  
Gen 8 Best: o3dpIest (6)  
Gen 9 Best: oodptert (7)  
Gen 10 Best: oodptert (7)  
Gen 11 Best: oodptest (8)  
Genome string evolved: oodptest  
Total Evolution Time: 0.007 seconds.
```

```

Gen 975 Best: supFrcalifrLgiliseiKexpbYRidociuy (25)
Gen 976 Best: supFrcalifrLgiliseisexpbYRidociuy (25)
Gen 977 Best: supFrcalifrkgiliseisexpbYRidociuy (25)
Gen 978 Best: supFrcalifrLgiliseisexpbYRidociuu (25)
Gen 979 Best: supFrcalifrkgiliseiKexpbYRidociuy (25)
Gen 980 Best: supFrcalifrLgiliseiKexpbYRidociuu (25)
Gen 981 Best: supFrcalifrLgiliseisexpbYRidociuy (25)
Gen 982 Best: supFrcalifrLgiliseisexpbYRidociuu (25)
Gen 983 Best: supFrcalifrLgiliseisexpbYRidociuy (25)
Gen 984 Best: supFrcalifrLgiliseiKexpbYRidociuu (25)
Gen 985 Best: supFrcalifrLgiliseisexpbYRidociuu (25)
Gen 986 Best: supFrcalifrLgiliseisexpbYRidociuy (25)
Gen 987 Best: supFrcalifrkgiliseiKexpbYRidociuy (25)
Gen 988 Best: supFrcalifrLgiliseisexpbYRidociuy (25)
Gen 989 Best: supFrcalifrLgiliseiKexpbYRidociuy (25)
Gen 990 Best: supFrcalifrLgiliseisexpbYRidociuu (25)
Gen 991 Best: supFrcalifrLgiliseiKexpbYRidociuu (25)
Gen 992 Best: supFrcalifrkgiliseisexpbYRidociuy (25)
Gen 993 Best: supFrcalifrkgiliseisexpbYRidociuu (25)
Gen 994 Best: supFrcalifrkgiliseisexpbYRidociuy (25)
Gen 995 Best: supFrcalifrkgiliseiKexpbYRidociuy (25)
Gen 996 Best: supFrcalifrLgiliseisexpbYRidociuu (25)
Gen 997 Best: supFrcalifrLgiliseisexpbYRidociuy (25)
Gen 998 Best: supFrcalifrLgiliseisexpbYRidociuu (25)
Gen 999 Best: supFrcalifrLgiliseiKexpbYRidociuy (25)
Total Evolution Time: 2.839 seconds.

```

Testing

Input	Output	TimeTaken(s)	Generation	Fitness Value
SRMKTR	SRMKTR	0.007	10	6
oodptest	oodptest	0.007	11	8
supercalifragilistic expialidocius	supFrcalifrLgiliseiKexpbYRidociuy	2.839	999	25

Gene Pool : 300 || Mutation Rate: 0.07 || Max Generations: 1000

Features and Enhancements:

- Can customize gene pool, mutation rate, max generations
- Future Improvements:
 - Tournament based selection instead of roulette base
 - Implementing elitism on each generation
 - Better UI and text formatting

Conclusion: The model performs exceptionally well upto 15 characters after which accuracy starts to decline.

- **Student Insight**
 - Learnt about Genetic algorithms and string evolution problem
 - Learnt about OODP concepts in context of genetic algorithms themselves
 - Console app window and time management
- **Importance of OOP and UML in development**
 - OOP helps making modular applications which can cater to different variety of same subset of tasks such as in case of genetic algorithms with OOP we can make genomes and populations which are more generalised to other problems not just to string evolutions
 - UML diagrams are essential to layout the development and manage it without stress and panic
 - They give a clear vision of what to implement what to expect of the outcome from respective input and outputs.

References:

Kie Codes YT: <https://www.youtube.com/watch?v=nhT56blfRpE>

Kie Codes YT: <https://www.youtube.com/watch?v=uQj5UNhCPuo>

Computerphile YT: <https://www.youtube.com/watch?v=MacVqujSXWE>

UltraEvolution: https://ultraevolution.org/wiki/string_guessing_tutorial/

GeeksForGeeks: <https://www.geeksforgeeks.org/dsa/genetic-algorithms/>

UML diagrams model file and code can be found on the public github repository:

<https://github.com/Vise-1/StringEvolution-using-GeneticAlgorithm>